



Iğdır Üniversitesi

Yazılım Mühendisliği Bölümü

Grafik Programlama Dersi Proje Raporu

**Proje Adı: Pygame ile Olay Yönelimli Asit-Baz Titrasyon
Simülasyonu**

Öğrenci: Polat Bilir

Tarih: Nisan 2026

İçindekiler

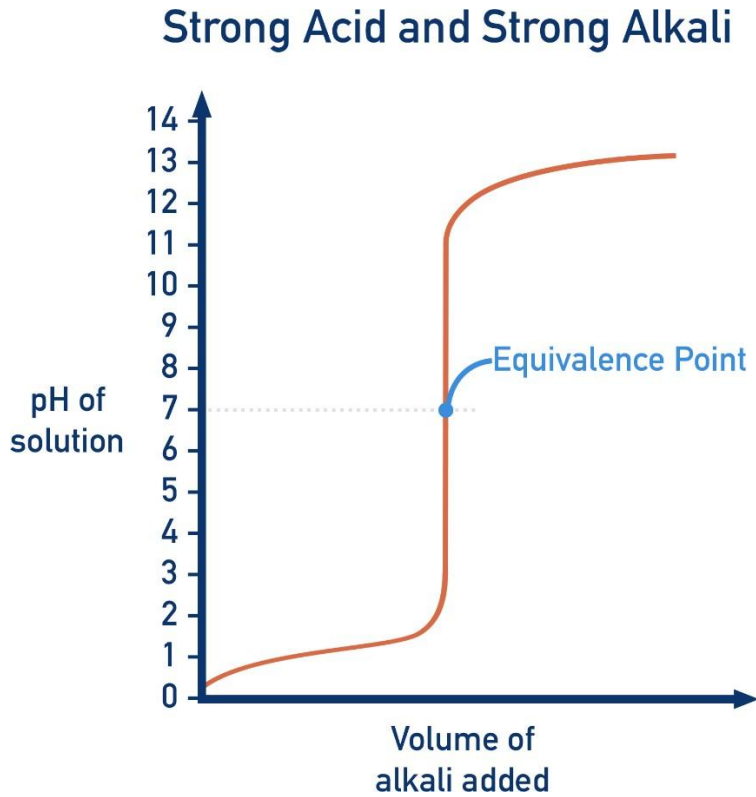
1. Projenin Amacı ve Kapsamı	3
2. Teorik Arka Plan ve Matematiksel Modelleme	3
3. Sistem Mimarisi ve Algoritma Tasarımı (OOP & EDA).....	4
4. Geliştirilen Ek Özellikler (Bonus Mekanikler)	5
5. Karşılaşılan Teknik Zorluklar ve Çözüm Yaklaşımları	5
6. Sonuç ve Test Ekranları.....	7
7. Kaynakça	10

1. Projenin Amacı ve Kapsamı

Bu projenin temel amacı, güçlü asit ve güçlü bazların titrasyon sürecini dijital ortamda yüksek doğrulukla modelleyen interaktif bir simülasyon geliştirmektir. Proje kapsamında Python programlama dili ve Pygame kütüphanesi kullanılmıştır. Geliştirme sürecinde; klasik donuk grafikler yerine olay yönelimli programlama (Event-Driven Programming) mantığıyla çalışan kullanıcı etkileşimli bir arayüz tasarlanması, görsel işleme mantığı ile matematiksel hesaplama mantığının birbirinden izole edildiği modüler bir mimari kurgulanması hedeflenmiştir.

2. Teorik Arka Plan ve Matematiksel Modelleme

Titrasyon, derişimi bilinen bir çözelti (titrant) yardımıyla derişimi bilinmeyen bir çözeltinin (analit) derişimini belirleme yöntemidir. Simülasyonumuzda, erlen içinde bulunan güçlü asit üzerine, büretten damla damla güçlü baz eklenmesi modellenmiştir.



Şekil 1: Güçlü Asit - Güçlü Baz Titrasyon Eğrisi

Simülasyonun kimya motoru, eklenen baz hacmine bağlı olarak ortamdaki pH değerini anlık olarak şu formüllerle hesaplamaktadır:

- Başlangıç Durumu: Sadece asidin bulunduğu durumdur.

$$pH = -\log_{10}(C_a)$$

- Eşdeğerlik Noktası Öncesi: Ortam hala asidiktir. Kalan hidrojen iyonu derişimi üzerinden hesaplanır.

$$[H^+] = \frac{C_a \cdot V_a - C_b \cdot V_b}{V_a + V_b}$$

$$pH = -\log_{10}([H^+])$$

- Eşdeğerlik Noktası: Güçlü asit-baz tepkimesinde nötralleşmenin tam gerçekleştiği andır.

$$pH = 7.0$$

- Eşdeğerlik Noktası Sonrası: Ortamda fazladan baz birikmeye başlar.

$$[OH^-] = \frac{C_b \cdot V_b - C_a \cdot V_a}{V_a + V_b}$$

$$pOH = -\log_{10}([OH^-])$$

3. Sistem Mimarisi ve Algoritma Tasarımı (OOP & EDA)

Projede kodun sürdürülebilirliğini artırmak amacıyla Nesne Yönelimli Programlama (ObjectOriented Programming) prensipleri benimsenmiştir. Sistem, sorumlulukların ayrılığı ilkesine göre üç ana sınıfa (Class) bölünmüştür:

1. **KimyaMotoru (Logic Layer):** Simülasyonun veri katmanıdır. Ekranda ne çizildiğinden bağımsız olarak sadece kimyasal formüllerin işletilmesinden ve indikatör renklerinin RGB formatında doğrusal interpolasyon (Lerp) ile harmanlanmasından sorumludur.

2. **ArayüzBilesenleri (Render Layer): Grafik** katmanıdır. Sadece büret, vana ve erlen gibi objelerin ekrana çizdirilmesi (pygame.draw metotları) görevini üstlenir.
3. **TitrasyonSimulasyonu (Controller Layer):** Ana denetleyicidir. Pygame olay döngüsünü (Event Loop) çalıştırır, FPS yönetimini sağlar ve kullanıcının fare/klavye girdilerini alarak sistemi koordine eder.

4. Geliştirilen Ek Özellikler (Bonus Mekanikler)

Proje yönergesinde belirtilen asgari istelere ek olarak (+20 puan hedefiyle), kullanıcı deneyimini zenginleştiren şu özellikler sisteme entegre edilmiştir:

1. **Gelişmiş Vana Kontrolü (Drag & Drop):** Yalnızca tıklama mekanizmasıyla yetinilmeyip, pygame.MOUSEMOTION olayı dinlenerek vananın fare ile sürüklenerek (sağa/sola) dinamik bir şekilde açılıp kapanması sağlanmıştır.
2. **Dinamik İndikatör Seçimi:** Kullanıcının klavye kısayollarını (1, 2, 3 tuşları) kullanarak simulasyon anında Fenolftalein, Metil Oranj ve Bromotimol Mavisi arasında anlık geçiş yapabilmeleri sağlanmıştır.

5. Karşılaşılan Teknik Zorluklar ve Çözüm Yaklaşımları

Zorluk 1 - Durum Yönetimi (State Management): Vananın fare ile sürüklenmesi sırasında kullanıcının vanayı tutup tutmadığının sistem tarafından anlık takibi gerekiyordu.

Çözüm 1: Olay döngüsünde (Event Loop) bir vana_surukleniyor durum bayrağı (boolean flag) oluşturuldu. Pygame'in sağladığı fare olayları dinlenerek durum makinesi mantığıyla akış kontrol altına alındı.

```
if event.type == pygame.MOUSEBUTTONDOWN and mesafe <= 25:
    self.vana_surukleniyor = True # Sürükleme başladı
elif event.type == pygame.MOUSEBUTTONUP:
    self.vana_surukleniyor = False # Sürükleme bitti
elif event.type == pygame.MOUSEMOTION and self.vana_surukleniyor:
    fark_x = event.pos[0] - self.vana_merkez[0]
    # Hızı farenin X eksenindeki konumuna göre dinamik ayarla
```

Şekil 2: Fare sürükleme durumunu kontrol eden olay döngüsü (Event Loop) mekanizması.

Zorluk 2 - Pürüzsüz Renk Geçişleri: İndikatör renklerinin pH değişimine göre sert bir şekilde değişmesi gerçekçi bir simülasyon hissi vermiyordu.

Çözüm 2: İki RGB dizisi arasında doğrusal interpolasyon (Lerp) yapan özel bir fonksiyon yazılarak kusursuz bir renk gradyanı oluşturuldu.

```
def _renk_karistir(self, renk1, renk2, oran):  
    # Oran değerini 0.0 ile 1.0 arasına sabitle  
    oran = max(0.0, min(1.0, oran))  
    r = int(renk1[0] * (1 - oran) + renk2[0] * oran)  
    g = int(renk1[1] * (1 - oran) + renk2[1] * oran)  
    b = int(renk1[2] * (1 - oran) + renk2[2] * oran)  
    return (r, g, b)
```

Şekil 3: İki RGB değeri arasında doğrusal interpolasyon (Lerp) sağlayan renk karıştırıcı.

Zorluk 3 - Dinamik Eksen Ölçeklendirmesi: Alt kısımda çizilen pH-Hacim grafiğinde, musluk açık kaldıkça eklenen baz hacmi sürekli arttığı için çizilen eğrinin ekrandan dışarı taşma riski mevcuttu.

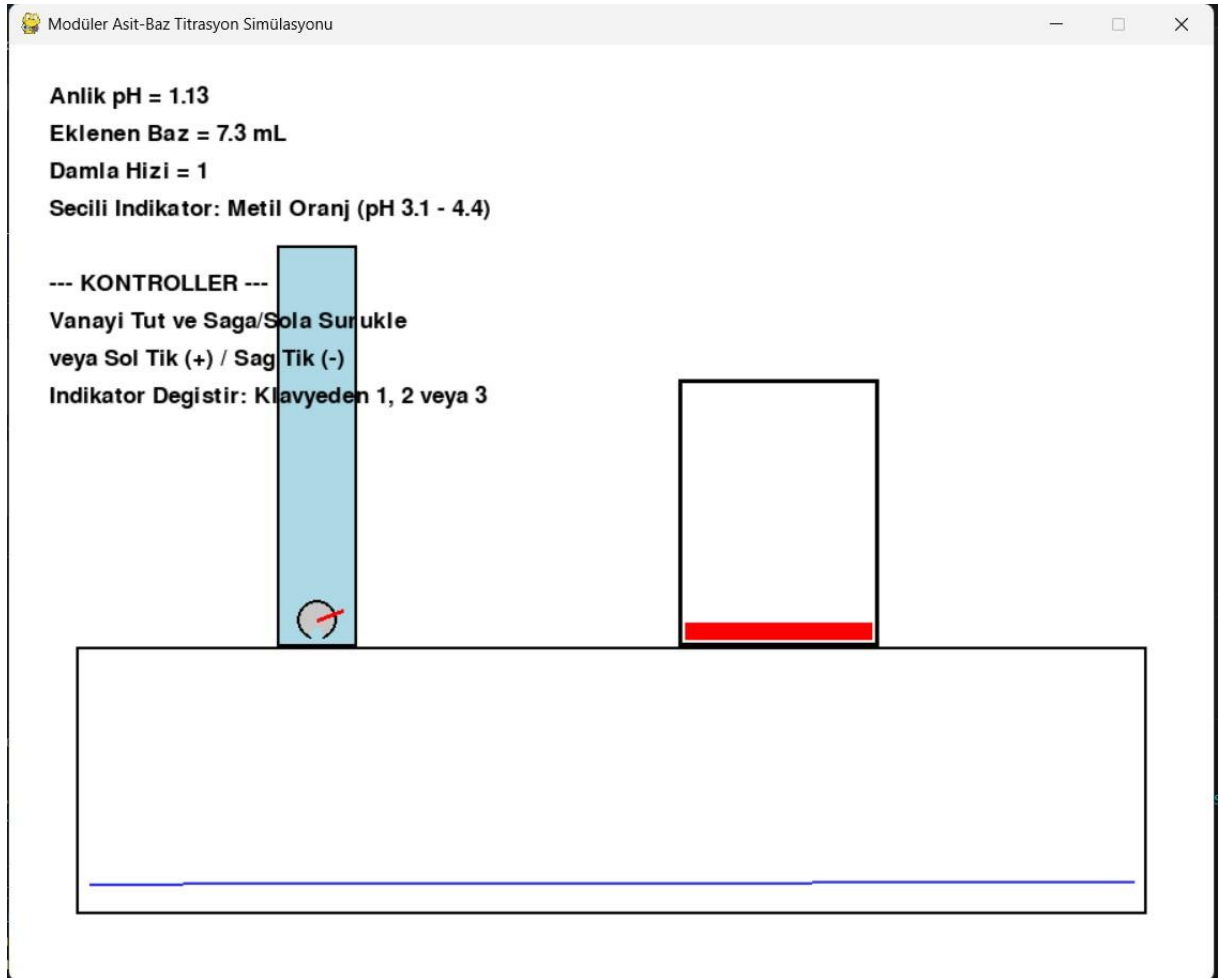
Çözüm 3: Toplanan veri seti içindeki maksimum hacim değeri her çevrimde denetlendi. Grafiğin X eksen koordinatları, anlık maksimum değere bölünerek normalize edildi.

```
# Maksimum hacmi bul (0'a bölünmeyi engellemek için minimum 1.0)  
max_hacim = max([v for v, p in self.veri_listesi]) if self.veri_listesi else 1.0  
  
# X eksen koordinatını (gx) anlık maksimum değere göre normalize et  
gx = x + 10 + (hacim / max_hacim) * (gen - 20)
```

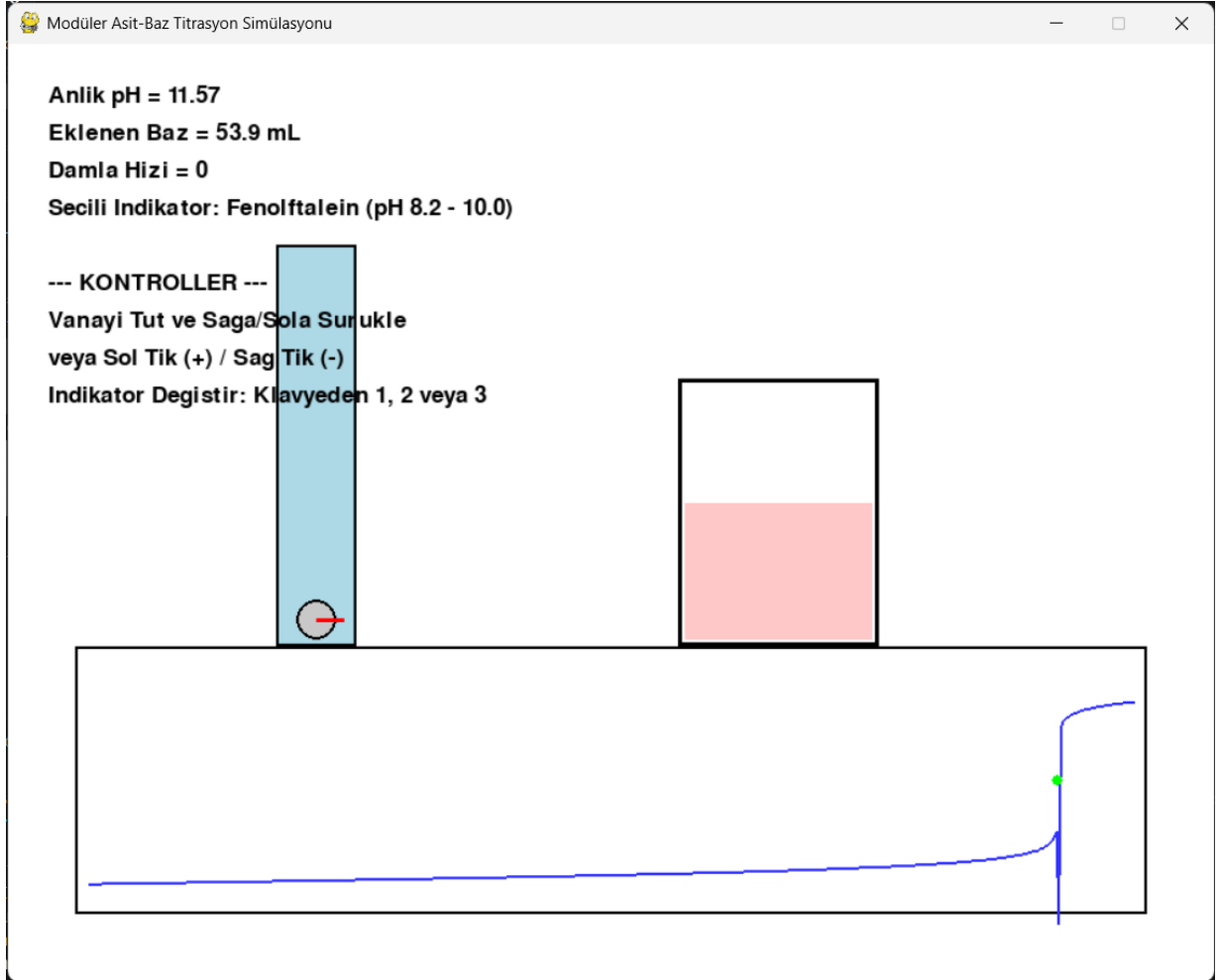
Şekil 4: X ekseninin dinamik ölçeklendirilmesi için normalize işlemi.

6. Sonuç ve Test Ekranları

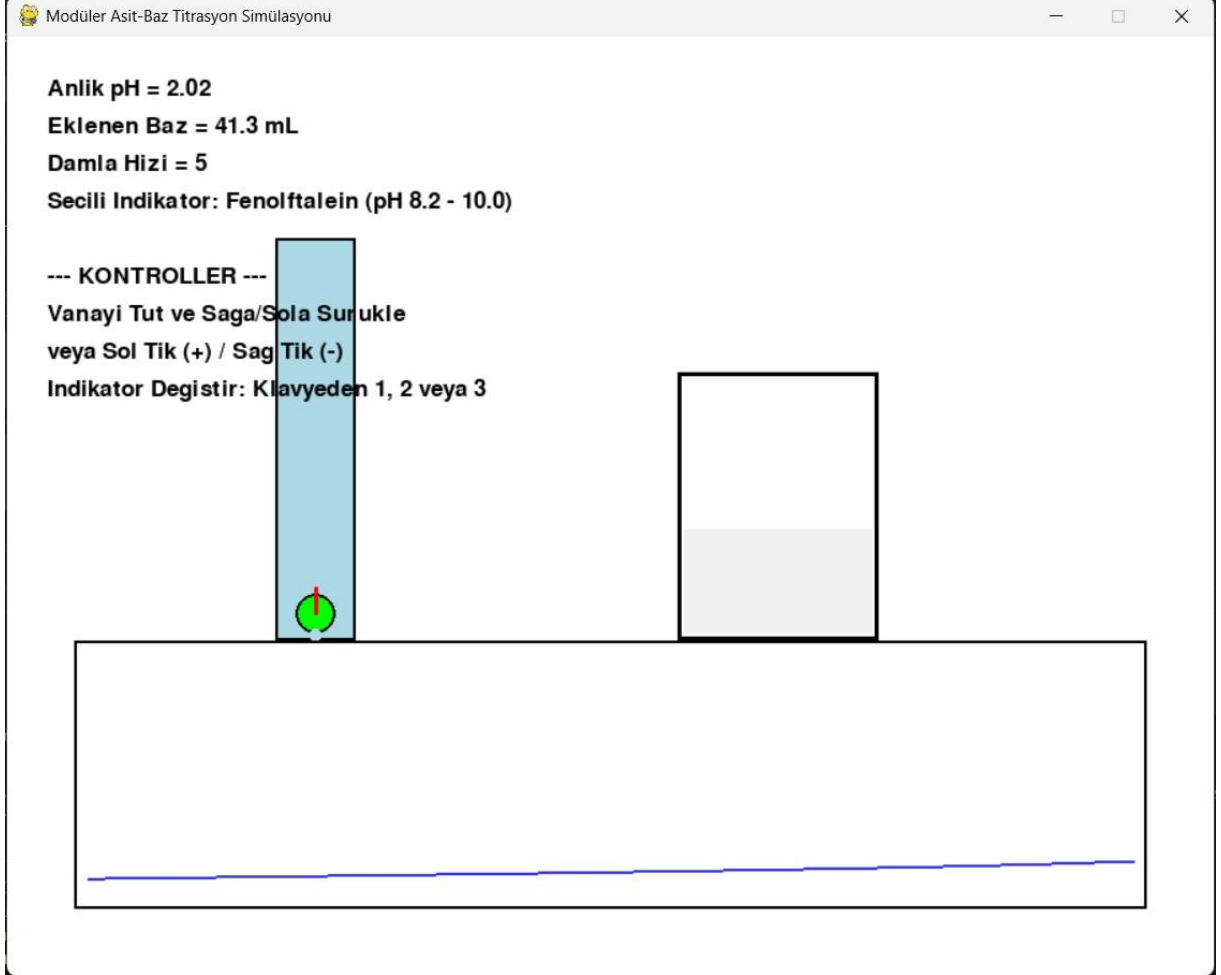
Geliştirilen simülasyon, belirtilen isterleri hatasız bir şekilde yerine getirmekte olup, modüler kod yapısı sayesinde genişletilmeye açık bir temel sunmaktadır. Aşağıda simülasyonun farklı aşamalarında alınan ekran görüntüleri yer almaktadır.



Şekil 5 : Simülasyonun başlangıcı, asidik ortam ve dinamik grafik çizimi.



Şekil 6: Eşdeğerlik noktasının aşılması ve indikatör renk değişimi.



Şekil 7: Bonus özellik olan fare ile vana (drag & drop) etkileşimi.

7. Kaynakça

Pygame Resmi Dokümantasyonu (pygame.org/docs)

Python 3 Standard Library Documentation

Grafik Programlama Dersi Asit-Baz Titrasyon Simülasyonu Ödev Dokümanı